REGLAT at AlexandriaX-2026: A Two-Stage Multi-Encoder Framework for Character-Level Error Detection and LQM Classification in Dialectal Arabic Machine Translation

Mariam Labib[1,2] **and Bakhom Ramzy**[1] **and Nsrin Ashraf**[1,3] **and Ahmed M. Fetouh**[3] and **Hamada Nayel**[3,4]
[1]Computer Engineering, Elsewedy University of Technology, Cairo, Egypt
[2]Faculty of Engineering, Mansoura University, Egypt
[3]Faculty of Computers and Artificial Intelligence, Benha University, Egypt
[4]College of Engineering,Wadi Ad Dwaser, Prince Sattam Bin Abdulaziz University, KSA

Abstract

We describe REGLAT's submission to AlexandriaX-2026 Subtask 3, which requires character-offset error spans in dialectal Arabic machine translations and a Linguistically Motivated Multidimensional Quality Metrics (LQM) category for each span. The system separates localization from classification. Four CAMeLBERT-DA and MARBERTv2 BIO taggers project token probabilities to a shared character grid; whitespace-segment probabilities are then thresholded, bridged, and optionally dilated. A second four-member ensemble classifies detected spans using the marked hypothesis, English source, and translation direction, with class weighting and fitted logit biases. REGLAT ranked fourth out of six teams with an official Overall Score of 0.4021 (Overlap F1 0.4272; Error-Class score 0.3770). Its Exact Match F1 of 0.0041 exposes severe boundary imprecision. We document a scorer mismatch in development-time selection and therefore treat earlier development values as proxy-metric results, not official-metric estimates.

1 Introduction

Fine-grained machine translation (MT) evaluation identifies and categorizes local errors instead of assigning only a sentence-level score. LLM-based approaches such as AutoMQM and GEMBA-MQM generate error spans and labels (Fernandes et al., 2023; Kocmi and Federmann, 2023); MQM-APE filters error annotations through automatic post-editing (Lu et al., 2025), while xCOMET learns both sentence-level quality and error spans (Guerreiro et al., 2024). Error Span Annotation provides a lighter human protocol (Kocmi et al., 2024), and WMT quality-estimation work includes fine-grained error prediction (Blain et al., 2023). Span extraction also has a long evaluation precedent in machine comprehension (Rajpurkar et al., 2016).

Arabic is one of the most widely spoken languages in the world, with hundreds of millions of speakers across the Arab region and beyond. However, automatic processing of Arabic remains a challenging task due to the complexity and richness of its morphology, the substantial variation among Arabic dialects, the absence of standardized resources for many dialectal varieties, and the limited availability of high-quality annotated datasets and computational resources (Ehab et al., 2026; Labib et al., 2026; Fetouh et al., 2026). These challenges have made Arabic Natural Language Processing (NLP) an active and important area of research. Dialectal Arabic adds variety- and register-sensitive phenomena that general MT evaluation can miss (Zbib et al., 2012; Alabdullah et al., 2026; Nowshin et al., 2026).

AlexandriaX-2026 Subtask 3 (El Mekki et al., 2026) adopts the six-level LQM taxonomy—graphetics, orthography, morphosyntax, semantics, pragmatics, and sociolinguistics—introduced by Magdy et al. (2026). We submit a two-stage system: an Arabic-hypothesis-only span detector followed by a source- and direction-conditioned classifier. Our main technical device projects probabilities from incompatible encoder tokenizations onto a shared character grid before decoding. The implementation is available in the project repository.[1]

This camera-ready report makes two corrections to the original analysis. First, development-time model selection used a local proxy scorer that differs from the official scorer; we no longer compare those development values directly with official test results. Second, the submitted dilation setting is presented as a plausible contributor to poor boundaries, not as a demonstrated causal finding.

[1]https://github.com/mfrancies90/AlexandriaX-2026--subtask3

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 842–847
October 28-29, 2026 ©2026 Association for Computational Linguistics

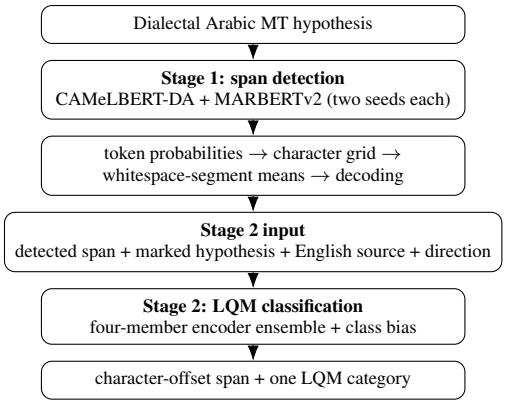


Figure 1: Submitted two-stage pipeline. The English source and direction enter Stage 2 only. Character probabilities are averaged over whitespace segments before Stage 1 decoding.

2 Task and Data

Each item contains an English source, a dialectal Arabic MT hypothesis, and a direction for Egyptian, Mauritanian, Moroccan, Palestinian, or Emirati Arabic. The released AlexandriaX Subtask 3 dataset (UBC-NLP, 2026) contains 1,125 training, 138 development, and 145 test items. Training has 1,997 spans; sociolinguistics accounts for 1,157 (57.9%), whereas graphetics has only two.

Predictions contain a start–end offset and one LQM category. The official implementation computes overlap from character-coverage masks and computes the class component from globally accumulated true positives, false positives, and false negatives, although the released field is named `class_macro_f1`. The ranking score is

$$\text{Overall} = \tfrac{1}{2}(\text{Overlap F1} + \text{Error-Class score}). \quad (1)$$

Exact Match F1 is diagnostic and is not included in Overall. NileChat-3B (El Mekki et al., 2025) is the shared-task baseline.

3 System

3.1 Stage 1: Span Detection

We fine-tune CAMeLBERT-DA (Inoue et al., 2021) and MARBERTv2 (Abdul-Mageed et al., 2021) as BIO taggers, with seeds 42 and 43 for each encoder. Tokens that overlap a gold span receive B for the first overlapping token and I thereafter; ordinary tokens receive O and special tokens are ignored. For member m and token j with offset $[s, e)$, the token error probability is $p_{m,j} = P(B) + P(I)$. We add this probability to every covered character and average contributions from all model–token pairs. The implementation then averages character probabilities within whitespace-delimited segments. Segments above threshold τ form runs; runs separated by at most γ segment indices are joined, and their boundaries are expanded by δ characters. Thus, the output uses character offsets but does not retain learned within-word resolution.

Official test metric	Score
Overall Score	0.4021
Exact Match F1	0.0041
Overlap F1	0.4272
Error-Class score	0.3770

Table 1: Official CodaBench results for REGLAT (rank 4 of 6). No proxy development value is mixed into this table.

3.2 Stage 2: LQM Classification

Stage 2 is trained on all 1,997 gold training spans, not predicted or out-of-fold spans. For a span $[s, e)$, the first sequence contains the direction and the hypothesis with « and » around the span; the second contains the span text and English source. Four sequence classifiers use the same two encoders and seeds. We average log-softmax outputs and add a per-class bias fitted by coordinate ascent. Training on clean gold spans creates a mismatch with inference on noisy detected spans; we include this in the limitations.

3.3 Training and Selection

Both stages use length 256, AdamW, learning rate 3×10^{-5}, batch size 16, six epochs, 10% warmup, weight decay 0.01, and gradient clipping at 1.0. Stage 1 uses square-root inverse-frequency BIO weights. Stage 2 uses capped inverse-frequency weights and label smoothing 0.05. Appendix A gives exact weights and preprocessing details.

The decoder search covered $\tau \in \{0.2, \ldots, 0.6\}$, $\delta \in \{0, \ldots, 4\}$, $\gamma \in \{1, 2, 3\}$, and three boundary-refinement modes. A two-fold procedure tuned on one development half and evaluated on the other. However, the notebook's local scorer greedily treats any positive span intersection as a match and averages class F1 over present categories. This is not the official character-mask/global-count scorer. Therefore, the search selected the submitted system using a proxy objective. Its official test result remains valid, but the development and variant values are not official-metric estimates (Appendix C).

4 Official Results and Diagnosis

Table 1 reports the only directly comparable evaluation. The system produced 223 spans for 145 test items. The very low Exact Match F1 identifies boundary precision as the dominant failure mode. The submitted decoder used $\tau = 0.20$, $\delta = 3$, $\gamma = 3$, and no refinement. Expanding every decoded run by three characters is consistent with exact-boundary errors, and the proxy scorer did not penalize excess covered characters as the official-mask scorer does. Nevertheless, without a matched held-out comparison under the official scorer, this evidence does not establish that dilation caused the failure or that the detector was under-confident. We therefore treat the previously named "dilation trap" as a hypothesis for future controlled evaluation.

The released annotations also expose a structural ceiling. Only 36.3% of training and 39.1% of development gold spans align with whitespace-segment boundaries. Under an oracle that may emit only unique, non-overlapping segment-aligned spans, the maximum possible Exact Match F1 is 0.527 on training and 0.558 on development, before classification is considered (Appendix B). This supports replacing whitespace decoding with a boundary-aware character or subword decoder.

5 Conclusion

REGLAT combines multi-encoder span detection with source-conditioned LQM classification and achieved an official Overall Score of 0.4021. The camera-ready analysis narrows the claim: the official result is valid, while development selection relied on a mismatched proxy scorer, and dilation is only a plausible boundary-error mechanism. The strongest next steps are to select parameters with the official scorer, preserve within-word boundary resolution, train Stage 2 on noisy or out-of-fold spans, and represent overlapping multi-category errors explicitly.

Limitations

The detector observes only the Arabic hypothesis, so semantic errors may be undetectable without the English source and sociolinguistic errors may require the target direction. We did not retain a controlled Stage 1 comparison of each encoder alone, so the experiments do not isolate the benefit of multi-encoder ensembling.

Stage 2 is trained on gold spans but applied to detected spans. It emits one category per detected interval, whereas the dataset contains overlapping and identical-offset annotations with different categories. Stage 1 converts overlaps to one binary BIO sequence; later annotations can overwrite earlier token labels. These design choices prevent exact reproduction of some gold structures.

Character probabilities are collapsed to whitespace segments before decoding. This removes within-word boundary evidence and sharply limits exact-span recovery. The development set has only 138 instances and no graphetics examples, making class-specific tuning unstable. Finally, cached submitted predictions were not available for rescoring; historical development numbers are retained only as proxy diagnostics.

References

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT & MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7088–7105, Online. Association for Computational Linguistics.

Abdullah Alabdullah, Lifeng Han, and Chenghua Lin. 2026. Ara-HOPE: Human-centric post-editing evaluation for dialectal Arabic to Modern Standard Arabic translation. In *Proceedings of the 13th Workshop on NLP for Similar Languages, Varieties and Dialects*, pages 157–171, Rabat, Morocco. Association for Computational Linguistics.

Frederic Blain, Chrysoula Zerva, Ricardo Rei, Nuno M. Guerreiro, Diptesh Kanojia, José G. C. de Souza, Beatriz Silva, Tânia Vaz, Yan Jingxuan, Fatemeh Azadi, Constantin Orasan, and André Martins. 2023. Findings of the WMT 2023 shared task on quality estimation. In *Proceedings of the Eighth Conference on Machine Translation*, pages 629–653, Singapore. Association for Computational Linguistics.

Rana Ehab, Ahmed El-Sawy, Mohammed Aldawsari, and Hamada Nayel. 2026. Deast: A dataset for english-arabic scientific translation and vice versa. *Data in Brief*, 64:112381.

Abdellah El Mekki, Houdaifa Atou, Omer Nacar, Shady Shehata, and Muhammad Abdul-Mageed. 2025. NileChat: Towards linguistically diverse and culturally aware LLMs for local communities. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 10967–10991, Suzhou, China. Association for Computational Linguistics.

Abdellah El Mekki, AbdelRahim A. Elmadany, Samar M. Magdy, Saad Ezzini, Mo El-Haj, Mustafa

Jarrar, Zaid Alyafeai, Bernard Ghanem, and Muhammad Abdul-Mageed. 2026. AlexandriaX 2026: The first shared task on dialectal Arabic machine translation. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Patrick Fernandes, Daniel Deutsch, Mara Finkelstein, Parker Riley, André Martins, Graham Neubig, Ankush Garg, Jonathan Clark, Markus Freitag, and Orhan Firat. 2023. The devil is in the errors: Leveraging large language models for fine-grained machine translation evaluation. In *Proceedings of the Eighth Conference on Machine Translation*, pages 1066–1083, Singapore. Association for Computational Linguistics.

Ahmed M. Fetouh, Mohammed Rahmath, Omer Dawood, Mariam Labib, Nsrin Ashraf, and Hamada Nayel. 2026. REGLAT at AbjadMed: Handling imbalanced Arabic medical text classification via hierarchical KNN-MLP architecture. In *Proceedings of the 2nd Workshop on NLP for Languages Using Arabic Script*, pages 394–397, Rabat, Morocco. Association for Computational Linguistics.

Nuno M. Guerreiro, Ricardo Rei, Daan van Stigt, Luisa Coheur, Pierre Colombo, and André F. T. Martins. 2024. xCOMET: Transparent machine translation evaluation through fine-grained error detection. *Transactions of the Association for Computational Linguistics*, 12:979–995.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 92–104, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Tom Kocmi and Christian Federmann. 2023. GEMBA-MQM: Detecting translation quality error spans with GPT-4. In *Proceedings of the Eighth Conference on Machine Translation*, pages 768–775, Singapore. Association for Computational Linguistics.

Tom Kocmi, Vilém Zouhar, Eleftherios Avramidis, Roman Grundkiewicz, Marzena Karpinska, Maja Popovic, Mrinmaya Sachan, and Mariya Shmatova. 2024. Error span annotation: A balanced approach for human evaluation of machine translation. In *Proceedings of the Ninth Conference on Machine Translation*, pages 1440–1453, Miami, Florida, USA. Association for Computational Linguistics.

Mariam Labib, Nsrin Ashraf, Ahmed M. Fetouh, and Hamada Nayel. 2026. REGLAT at AbjadGenEval: Multi-model ensemble approach for Arabic AI-generated text detection. In *Proceedings of the 2nd Workshop on NLP for Languages Using Arabic Script*, pages 493–496, Rabat, Morocco. Association for Computational Linguistics.

Qingyu Lu, Liang Ding, Kanjian Zhang, Jinxia Zhang, and Dacheng Tao. 2025. MQM-APE: Toward high-quality error annotation predictors with automatic post-editing in LLM translation evaluators. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 5570–5587, Abu Dhabi, UAE. Association for Computational Linguistics.

Samar M. Magdy, Fakhraddin Alwajih, Abdellah El Mekki, Wesam El Sayed, and Muhammad Abdul-Mageed. 2026. LQM: Linguistically motivated multidimensional quality metrics for machine translation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 40470–40493, San Diego, California, United States. Association for Computational Linguistics.

Afroza Nowshin, Prithweeraj Acharjee Porag, Haziq Jeelani, and Fayeq Jeelani Syed. 2026. Context-aware dialectal Arabic machine translation with interactive region and register selection. *Preprint*, arXiv:2604.06456.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.

UBC-NLP. 2026. AlexandriaX subtask 3 dataset. Hugging Face Datasets. Accessed 2026-09-03.

Rabih Zbib, Erika Malchiodi, Jacob Devlin, David Stallard, Spyros Matsoukas, Richard Schwartz, John Makhoul, Omar F. Zaidan, and Chris Callison-Burch. 2012. Machine translation of Arabic dialects. In *Proceedings of the 2012 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 49–59, Montréal, Canada. Association for Computational Linguistics.

A Reproducibility Details

The exact checkpoints are `CAMeL-Lab/bert-base-arabic-camelbert-da` and `UBC-NLP/MARBERTv2`.

In the taxonomy order used in Section 1—graphetics, orthography/writing conventions, morphosyntax, semantics, pragmatics, and sociolinguistics—the normalized Stage 2 class-weight vector was $(4.267, 0.536, 0.346, 0.117, 0.684, 0.049)$, and the fitted class-bias vector was $(-2.00, 0.00, 1.25, 0.00, -0.25, 0.00)$. This is a reordering for presentation only; the implementation's internal label order was graphetics, morphosyntax, orthography/writing conventions, pragmatics, semantics, and sociolinguistics.

Setting	Value
Checkpoints	CAMeLBERT-DA; MARBERTv2
Seeds	42, 43 per encoder and stage
Members	4 per stage
Maximum length	256
Optimizer	AdamW; weight decay 0.01
Learning rate / batch	3×10^{-5} / 16
Epochs / warmup	6 / 10% linear
Gradient clipping	1.0
Stage 1 weights (O/B/I)	0.424 / 1.433 / 1.143
Stage 2 smoothing	0.05
Mixed precision	CUDA autocast (FP16 on T4)
Checkpoint choice	final epoch; no best-dev checkpoint

Table 2: Submitted training configuration reconstructed from the public notebook and its recorded output.

Audit statistic	Train	Dev
Items	1,125	138
Gold spans	1,997	243
Boundary-aligned spans	725	95
Boundary-alignment rate	36.3%	39.1%
Rows with overlapping spans	35	4
Overlapping span pairs	52	5
Identical-offset pairs	9	1
Adjacent span pairs	34	3
Oracle non-overlap exact TP	714	94
Oracle Exact Match F1	0.527	0.558

Table 3: Audit of the released annotations. Overlap counts include identical offsets. The oracle allows only unique, non-overlapping spans whose boundaries coincide with whitespace segments and assumes perfect precision; it is a representation ceiling, not a model score.

No Arabic normalization, Unicode normalization, or diacritic removal is performed. Offsets are Python string indices, i.e., Unicode code-point positions in the execution environment, not byte offsets or grapheme-cluster indices. Fast-tokenizer offset mappings project gold spans to subwords. Special tokens have empty offsets and label -100. Inference sums $P(B)+P(I)$, so it neither decodes nor repairs BIO transitions.

The «/» span markers are ordinary text, not added special tokens. Stage 1 uses the tokenizer's default right truncation. Stage 2 uses `longest_first` pair truncation. A tokenizer audit found Stage 1 truncation in 2/1 training items for CAMeLBERT-DA/MARBERTv2 and none in development. For Stage 2, 40/3 of 1,997 training pairs and 1/0 of 243 development pairs exceeded 256 tokens; all gold-span content remained, although one training pair lost its closing marker for each tokenizer.

B Representation Audit

Stage 1 loops through annotations and assigns BIO labels to every overlapping token. When annotations overlap, a later annotation may overwrite labels assigned by an earlier annotation. Adjacent predicted segments can also be joined by the gap rule. Stage 2 retains each gold span as a separate training item, including duplicate offsets with different labels, but inference assigns only one category to each detected interval. A multi-label or structured span representation is required to reproduce these cases.

We cannot isolate an overlap-specific change in Exact Match or Error-Class score from the retained artifacts: cached per-instance submitted predictions are unavailable, and the historical development matcher is the proxy scorer described in Appendix C. The counts above establish representational incompatibility, but they are not a model-effect estimate; a valid delta requires rescoring the same predictions with and without overlap-affected items under the official scorer.

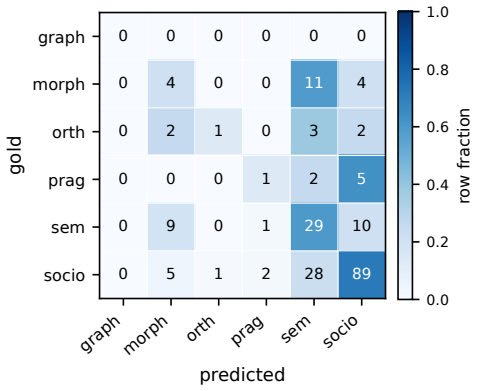


Figure 2: Row-normalized confusion matrix on the 209 development spans matched by the historical proxy procedure. Cell annotations are raw counts. This descriptive Stage 2 analysis is not the official Error-Class score and is not compared with test results.

C Development-Scorer Audit

The local scorer greedily pairs predicted and gold spans whenever their intersection is positive, then counts matched spans for overlap F1. It computes a per-category F1 and averages only categories present in gold or predictions. The official scorer instead uses character coverage masks for overlap and global class counts. Because selection and

Category	P	R	F1	n
Graphetics	0.00	0.00	0.00	0
Morphosyntax	0.20	0.21	0.21	19
Orthography	0.50	0.13	0.20	8
Pragmatics	0.25	0.13	0.17	8
Semantics	0.40	0.59	0.48	49
Sociolinguistics	0.81	0.71	0.76	125
Macro (six classes)			0.30	209

Table 4: Per-category performance on the same proxy-matched development spans as Figure 2; accuracy is 0.593. These are descriptive classifier diagnostics, not official-metric estimates.

Historical proxy result	Value
Two-fold held-out Overall	0.4180
Full-development Overall	0.4369
Full-development Exact Match F1	0.0044
Full-development overlap proxy	0.6346
Full-development class proxy	0.2392

Table 5: Values produced by the mismatched local proxy scorer. They are preserved for reproducibility but must not be compared numerically with Table 1.

logit-bias fitting used the local function, replacing the function after submission would define a post-submission system. We therefore report the submitted configuration and its official test score, label all old development values as proxy results, and avoid unsupported development-to-test explanations.